Running Data Visualisation

Annotated Source Code

CM1010 Introduction to Programming II
Ryo Nakayama (250151349) — March 2026

Table of Contents

1. **index.html** — HTML entry point — script loading order and own-code markers
2. **sketch.js** — Main sketch — Gallery registration of all 7 visualisations
3. **helper-functions.js** — Shared pace helper functions (own code: lines 42-63)
4. **tooltip.js** — Reusable tooltip component (composition pattern)
5. **bar-chart.js** — Reusable bar chart constructor
6. **scatter-plot.js** — Reusable scatter plot constructor with bubble chart support
7. **monthly-distance.js** — Monthly Distance — bar chart of distance per month
8. **pace-progress.js** — Pace Progress — moving average and pace zone backgrounds
9. **activity-types.js** — Activity Types — pie chart with dropdown switcher
10. **heartrate-vs-pace.js** — Heart Rate vs Pace — scatter plot with statistical analysis
11. **goal-tracker.js** — Goal Tracker — semicircular gauge with trigonometry
12. **training-heatmap.js** — Training Heatmap — GitHub-style calendar grid
13. **marathon-radar.js** — Marathon Readiness Radar — spider chart with 5 normalised metrics
14. **tests.js** — Unit and integration tests (TestRunner, black-box testing)

1. index.html

HTML entry point — script loading order and own-code markers

```html
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="utf-8">
    <meta http-equiv="X-UA-Compatible" content="IE=edge">
    <meta name="viewport" content="width=device-width, initial-scale=1">
    <title>Running Data Visualisation</title>

    <!-- Libraries -->
    <script src="lib/p5.min.js"></script>

    <link rel="stylesheet" href="style.css">

    <!-- Main sketch file -->
    <script src="sketch.js"></script>

    <!-- Helper functions and core components (from template) -->
    <script src="helper-functions.js"></script>
    <script src="gallery.js"></script>
    <script src="pie-chart.js"></script>

    <!-- Start of my own code -->
    <!-- New reusable chart constructors -->
    <script src="tooltip.js"></script>
    <script src="bar-chart.js"></script>
    <script src="scatter-plot.js"></script>

    <!-- New running data visualisations -->
    <script src="monthly-distance.js"></script>
    <script src="pace-progress.js"></script>
    <script src="activity-types.js"></script>
    <script src="heartrate-vs-pace.js"></script>
    <script src="goal-tracker.js"></script>
    <script src="training-heatmap.js"></script>
    <script src="marathon-radar.js"></script>
    <!-- End of my own code -->
  </head>
  <body>
    <div id="app" class="container">
      <ul id="visuals-menu"></ul>
    </div>
  </body>
</html>
```

2. sketch.js

Main sketch — Gallery registration of all 7 visualisations

```

// Global variable to store the gallery object. The gallery object is
// a container for all the visualisations.
var gallery;

function setup() {
  // Create a canvas to fill the content div from index.html.
  var c = createCanvas(1024, 576);
  c.parent('app');

  // Create a new gallery object.
  gallery = new Gallery();

  /* Start of my own code */
  // Add running data visualisations
  gallery.addVisual(new MonthlyDistance());
  gallery.addVisual(new PaceProgress());
  gallery.addVisual(new ActivityTypes());
  gallery.addVisual(new HeartRateVsPace());
  gallery.addVisual(new GoalTracker());
  gallery.addVisual(new TrainingHeatmap());
  gallery.addVisual(new MarathonRadar());
  /* End of my own code */
}

function draw() {
  background(255);
  if (gallery.selectedVisual != null) {
    gallery.selectedVisual.draw();
  }
}
```

3. helper-functions.js

Shared pace helper functions (own code: lines 42-63)

> *Lines 1–41: template code (not shown)*

```
42  // ---------------------------------------------------------------------
43  // Pace helper functions (shared across visualisations)
44  // ---------------------------------------------------------------------
45
46  // Converts pace string "6:30" to decimal minutes (6.5)
47  function paceToMinutes(paceStr) {
48    var parts = paceStr.split(':');
49    if (parts.length == 2) {
50      return parseInt(parts[0]) + parseInt(parts[1]) / 60;
51    }
52    return 0;
53  }
54
55  // Converts decimal minutes (6.5) to pace string "6:30"
56  function formatPace(paceMinutes) {
57    var minutes = Math.floor(paceMinutes);
58    var seconds = Math.round((paceMinutes - minutes) * 60);
59    if (seconds < 10) {
60      return minutes + ':0' + seconds;
61    }
62    return minutes + ':' + seconds;
63  }
```

> *Lines 64–154: template code (not shown)*

4. tooltip.js

Reusable tooltip component (composition pattern)

```
/* Start of my own code */
// Reusable tooltip component using composition pattern.
// Extracted from ScatterPlot to enable reuse across visualisations.
function Tooltip() {
  this.width = 120;
  this.padding = 5;
  this.lineHeight = 15;
  this.cornerRadius = 5;

  this.draw = function(x, y, lines, layout) {
    var tooltipHeight = this.lineHeight * lines.length + this.padding * 2;
    var tooltipX = x + 10;
    var tooltipY = y - tooltipHeight - 10;

    // Boundary detection: reposition if tooltip would go off-screen
    if (layout) {
      if (tooltipX + this.width > layout.rightMargin) {
        tooltipX = x - this.width - 10;
      }
      if (tooltipY < layout.topMargin) {
        tooltipY = y + 10;
      }
    }

    // Background
    fill(255, 255, 255, 230);
    stroke(100);
    strokeWeight(1);
    rect(tooltipX, tooltipY, this.width, tooltipHeight, this.cornerRadius);

    // Text lines
    fill(0);
    noStroke();
    textAlign('left', 'top');
    textSize(10);

    for (var i = 0; i < lines.length; i++) {
      text(lines[i], tooltipX + this.padding, tooltipY + this.padding + (i * this.lineHeight));
    }
  };
}
/* End of my own code */
```

5. bar-chart.js

Reusable bar chart constructor

```
/* Start of my own code */
// BarChart: Reusable bar chart constructor that dynamically calculates
// bar width based on data length and plot area. Accepts layout, data
// array, labels, and colour parameters.
function BarChart(layout) {
  this.layout = layout;
  this.barWidth = 30;
  this.barGap = 10;

  this.draw = function(data, labels, colours, title) {
    if (data.length == 0) {
      console.log('Data has length zero!');
      return;
    }

    if (labels.length != data.length) {
      console.log('Data and labels arrays must be the same length!');
      return;
    }

    var plotWidth = this.layout.rightMargin - this.layout.leftMargin;
    var plotHeight = this.layout.bottomMargin - this.layout.topMargin;
    this.barWidth = (plotWidth - (data.length + 1) * this.barGap) / data.length;
    var maxValue = max(data);

    if (title) {
      noStroke();
      fill(0);
      textAlign('center', 'center');
      textSize(16);
      text(title,
           (this.layout.leftMargin + this.layout.rightMargin) / 2,
           this.layout.topMargin - 20);
    }

    for (var i = 0; i < data.length; i++) {
      var x = this.layout.leftMargin + this.barGap + i * (this.barWidth + this.barGap);
      var barHeight = map(data[i], 0, maxValue, 0, plotHeight);
      var y = this.layout.bottomMargin - barHeight;

      if (Array.isArray(colours)) {
        fill(colours[i % colours.length]);
      } else {
        fill(colours);
      }

      stroke(0);
      strokeWeight(1);
      rect(x, y, this.barWidth, barHeight);

      noStroke();
      fill(0);
      textAlign('center', 'bottom');
      textSize(10);
      text(data[i].toFixed(1), x + this.barWidth / 2, y - 5);

      push();
      translate(x + this.barWidth / 2, this.layout.bottomMargin + 10);
      rotate(radians(-45));
      textAlign('right', 'center');
      textSize(10);
```

```
      text(labels[i], 0, 0);
      pop();
    }
  };

  this.drawYAxisTickLabels = function(minValue, maxValue, numTicks, decimalPlaces) {
    var range = maxValue - minValue;
    var tickStep = range / numTicks;

    fill(0);
    noStroke();
    textAlign('right', 'center');
    textSize(12);

    for (var i = 0; i <= numTicks; i++) {
      var value = minValue + (i * tickStep);
      var y = map(value, minValue, maxValue,
                  this.layout.bottomMargin,
                  this.layout.topMargin);

      text(value.toFixed(decimalPlaces),
           this.layout.leftMargin - 10,
           y);

      stroke(220);
      strokeWeight(1);
      line(this.layout.leftMargin, y,
           this.layout.rightMargin, y);
    }
  };
}
/* End of my own code */
```

6. scatter-plot.js

Reusable scatter plot constructor with bubble chart support

```
/* Start of my own code */
// ScatterPlot: Reusable scatter plot constructor that renders data points
// on a 2D plane. Uses composition with a Tooltip instance for hover
// interaction. Accepts layout, data arrays, and axis ranges as parameters.
function ScatterPlot(layout) {
  this.layout = layout;
  this.pointSize = 8;
  this.tooltip = new Tooltip();

  // sizes: optional array of per-point diameters (bubble chart mode)
  this.draw = function(xData, yData, xRange, yRange, colour, labels, sizes) {
    if (xData.length == 0 || yData.length == 0) {
      console.log('Data has length zero!');
      return;
    }

    if (xData.length != yData.length) {
      console.log('X and Y data arrays must be the same length!');
      return;
    }

    for (var i = 0; i < xData.length; i++) {
      var x = this.mapToX(xData[i], xRange.min, xRange.max);
      var y = this.mapToY(yData[i], yRange.min, yRange.max);
      var currentSize = (sizes && i < sizes.length) ? sizes[i] : this.pointSize;
      var isHovered = dist(mouseX, mouseY, x, y) < currentSize;

      if (isHovered) {
        fill(255, 100, 100);
        noStroke();
        ellipse(x, y, currentSize * 1.5, currentSize * 1.5);

        // Build tooltip lines using shared formatPace from helper-functions.js
        var lines = [
          'HR: ' + xData[i].toFixed(0) + ' bpm',
          'Pace: ' + formatPace(yData[i]) + '/km'
        ];
        if (labels && labels[i]) {
          lines.push(labels[i]);
        }
        this.tooltip.draw(x, y, lines, this.layout);
      } else {
        fill(colour);
        stroke(0);
        strokeWeight(0.5);
        ellipse(x, y, currentSize, currentSize);
      }
    }
  };

  this.mapToX = function(value, minVal, maxVal) {
    return map(value, minVal, maxVal,
               this.layout.leftMargin,
               this.layout.rightMargin);
  };

  this.mapToY = function(value, minVal, maxVal) {
    return map(value, minVal, maxVal,
               this.layout.bottomMargin,
               this.layout.topMargin);
  };
```

```

  this.drawXAxisTicks = function(minVal, maxVal, numTicks, label) {
    var range = maxVal - minVal;
    var step = range / numTicks;

    fill(0);
    noStroke();
    textAlign('center', 'top');
    textSize(10);

    for (var i = 0; i <= numTicks; i++) {
      var value = minVal + (i * step);
      var x = this.mapToX(value, minVal, maxVal);

      stroke(0);
      line(x, this.layout.bottomMargin, x, this.layout.bottomMargin + 5);

      noStroke();
      text(value.toFixed(0), x, this.layout.bottomMargin + 8);

      stroke(220);
      line(x, this.layout.topMargin, x, this.layout.bottomMargin);
    }

    textSize(12);
    text(label,
         (this.layout.leftMargin + this.layout.rightMargin) / 2,
         this.layout.bottomMargin + 30);
  };

  this.drawYAxisTicks = function(minVal, maxVal, numTicks, label) {
    var range = maxVal - minVal;
    var step = range / numTicks;

    fill(0);
    noStroke();
    textAlign('right', 'center');
    textSize(10);

    for (var i = 0; i <= numTicks; i++) {
      var value = minVal + (i * step);
      var y = this.mapToY(value, minVal, maxVal);

      text(formatPace(value), this.layout.leftMargin - 8, y);

      stroke(220);
      line(this.layout.leftMargin, y, this.layout.rightMargin, y);
    }

    push();
    translate(this.layout.leftMargin - 45, height / 2);
    rotate(-PI / 2);
    textAlign('center', 'center');
    textSize(12);
    text(label, 0, 0);
    pop();
  };
}
/* End of my own code */
```

7. monthly-distance.js

Monthly Distance — bar chart of distance per month

```
/* Start of my own code */
// Monthly Distance: Bar chart showing total running distance per month.
// Aggregates 185 activity records by year-month using object-based grouping,
// then delegates rendering to a reusable BarChart constructor.
function MonthlyDistance() {
  this.name = 'Monthly Distance';
  this.id = 'monthly-distance';
  this.loaded = false;

  var marginSize = 35;
  this.layout = {
    marginSize: marginSize,
    leftMargin: marginSize * 2,
    rightMargin: width - marginSize,
    topMargin: marginSize * 2,
    bottomMargin: height - marginSize * 3,
    pad: 5,
    plotWidth: function() {
      return this.rightMargin - this.leftMargin;
    },
    plotHeight: function() {
      return this.bottomMargin - this.topMargin;
    },
    grid: true,
    numYTickLabels: 6
  };

  this.barChart = new BarChart(this.layout);

  this.preload = function() {
    var self = this;
    this.data = loadTable(
      './data/running/activities.csv', 'csv', 'header',
      function(table) {
        self.loaded = true;
      });
  };

  this.setup = function() {
    if (!this.loaded) {
      console.log('Data not yet loaded');
      return;
    }
    this.monthlyData = this.aggregateByMonth();
  };

  this.destroy = function() {};

  this.aggregateByMonth = function() {
    var months = {};

    for (var i = 0; i < this.data.getRowCount(); i++) {
      var dateStr = this.data.getString(i, 'date');
      var distance = this.data.getNum(i, 'distance');
      var yearMonth = dateStr.substring(0, 7);

      if (months[yearMonth]) {
        months[yearMonth] += distance;
      } else {
        months[yearMonth] = distance;
      }
```

```
    }

    var sortedMonths = Object.keys(months).sort();
    var labels = [];
    var values = [];

    for (var j = 0; j < sortedMonths.length; j++) {
      var ym = sortedMonths[j];
      var parts = ym.split('-');
      var monthNames = ['Jan', 'Feb', 'Mar', 'Apr', 'May', 'Jun',
                        'Jul', 'Aug', 'Sep', 'Oct', 'Nov', 'Dec'];
      var monthLabel = monthNames[parseInt(parts[1]) - 1] + ' ' + parts[0].substring(2);
      labels.push(monthLabel);
      values.push(months[ym]);
    }

    return { labels: labels, values: values };
  };

  this.draw = function() {
    if (!this.loaded) {
      console.log('Data not yet loaded');
      return;
    }

    fill(0);
    noStroke();
    textSize(20);
    textAlign('center', 'center');
    text('Monthly Running Distance',
         (this.layout.leftMargin + this.layout.rightMargin) / 2,
         this.layout.topMargin / 2);

    push();
    translate(this.layout.leftMargin / 3, height / 2);
    rotate(-PI / 2);
    textSize(14);
    text('Distance (km)', 0, 0);
    pop();

    stroke(0);
    strokeWeight(1);
    line(this.layout.leftMargin, this.layout.topMargin,
         this.layout.leftMargin, this.layout.bottomMargin);
    line(this.layout.leftMargin, this.layout.bottomMargin,
         this.layout.rightMargin, this.layout.bottomMargin);

    var maxDistance = max(this.monthlyData.values);
    this.barChart.drawYAxisTickLabels(0, maxDistance, this.layout.numYTickLabels, 0);

    var barColour = color(66, 133, 244);
    this.barChart.draw(
      this.monthlyData.values,
      this.monthlyData.labels,
      barColour,
      null
    );

    var totalDistance = this.monthlyData.values.reduce(function(a, b) {
      return a + b;
    }, 0);

    textSize(12);
    textAlign('left', 'top');
    fill(100);
    text('Total Distance: ' + totalDistance.toFixed(1) + ' km',
```

```
          this.layout.leftMargin,
          this.layout.bottomMargin + 60);
    };
  }
  /* End of my own code */
```

8. pace-progress.js

Pace Progress — moving average and pace zone backgrounds

```
/* Start of my own code */
// Pace Progress: Displays running pace over time with individual data points
// and a 10-run trailing moving average trend line. Filters runs by minimum
// distance (3 km) and valid pace range (4–10 min/km) to ensure data quality.
function PaceProgress() {
  this.name = 'Pace Progress';
  this.id = 'pace-progress';
  this.title = 'Running Pace Progress Over Time';
  this.xAxisLabel = 'Date';
  this.yAxisLabel = 'Pace (min/km)';

  var marginSize = 35;
  this.layout = {
    marginSize: marginSize,
    leftMargin: marginSize * 2,
    rightMargin: width - marginSize,
    topMargin: marginSize * 2,
    bottomMargin: height - marginSize * 2,
    pad: 5,
    plotWidth: function() {
      return this.rightMargin - this.leftMargin;
    },
    plotHeight: function() {
      return this.bottomMargin - this.topMargin;
    },
    grid: true,
    numXTickLabels: 8,
    numYTickLabels: 8
  };

  this.loaded = false;

  this.preload = function() {
    var self = this;
    this.data = loadTable(
      './data/running/activities.csv', 'csv', 'header',
      function(table) {
        self.loaded = true;
      });
  };

  this.setup = function() {
    if (!this.loaded) {
      console.log('Data not yet loaded');
      return;
    }

    this.processedData = this.processData();
    this.startDate = this.processedData.dates[0];
    this.endDate = this.processedData.dates[this.processedData.dates.length - 1];
    this.minPace = Math.floor(min(this.processedData.paces)) - 0.5;
    this.maxPace = Math.ceil(max(this.processedData.paces)) + 0.5;
  };

  this.destroy = function() {};

  this.processData = function() {
    var dates = [];
    var paces = [];
    var distances = [];

```

```
      for (var i = 0; i < this.data.getRowCount(); i++) {
        var activityType = this.data.getString(i, 'activity_type');

        if (activityType === 'Run') {
          var dateStr = this.data.getString(i, 'date');
          var paceStr = this.data.getString(i, 'avg_pace');
          var distance = this.data.getNum(i, 'distance');

          if (distance >= 3) {
            var paceMinutes = paceToMinutes(paceStr);

            if (paceMinutes >= 4 && paceMinutes <= 10) {
              dates.push(new Date(dateStr));
              paces.push(paceMinutes);
              distances.push(distance);
            }
          }
        }
      }

      var combined = [];
      for (var j = 0; j < dates.length; j++) {
        combined.push({
          date: dates[j],
          pace: paces[j],
          distance: distances[j]
        });
      }
      combined.sort(function(a, b) {
        return a.date - b.date;
      });

      var sortedDates = [];
      var sortedPaces = [];
      var sortedDistances = [];
      for (var k = 0; k < combined.length; k++) {
        sortedDates.push(combined[k].date);
        sortedPaces.push(combined[k].pace);
        sortedDistances.push(combined[k].distance);
      }

      return {
        dates: sortedDates,
        paces: sortedPaces,
        distances: sortedDistances
      };
    };

    // Moving average smooths out short-term fluctuations.
    // O(n * w) time complexity where n = data length, w = window size.
    this.calculateMovingAverage = function(data, windowSize) {
      var result = [];
      for (var i = 0; i < data.length; i++) {
        var start = Math.max(0, i - windowSize + 1);
        var sum = 0;
        var count = 0;
        for (var j = start; j <= i; j++) {
          sum += data[j];
          count++;
        }
        result.push(sum / count);
      }
      return result;
    };

    // Draw colour-coded background bands for pace zones
```

```
    this.drawPaceZones = function() {
      noStroke();
      var plotLeft = this.layout.leftMargin;
      var plotWidth = this.layout.rightMargin - this.layout.leftMargin;

      var zones = [
        { min: this.minPace, max: 6, col: [144, 238, 144, 30], label: 'Fast' },
        { min: 6, max: 7, col: [255, 236, 153, 30], label: 'Moderate' },
        { min: 7, max: this.maxPace, col: [255, 180, 150, 30], label: 'Easy' }
      ];

      for (var i = 0; i < zones.length; i++) {
        var zone = zones[i];
        if (zone.min >= this.maxPace || zone.max <= this.minPace) continue;

        var clampedMin = Math.max(zone.min, this.minPace);
        var clampedMax = Math.min(zone.max, this.maxPace);
        var yTop = this.mapPaceToHeight(clampedMin);
        var yBottom = this.mapPaceToHeight(clampedMax);

        fill(zone.col[0], zone.col[1], zone.col[2], zone.col[3]);
        rect(plotLeft, yTop, plotWidth, yBottom - yTop);

        fill(100, 80);
        textSize(9);
        textAlign('right', 'center');
        text(zone.label, this.layout.rightMargin - 5, (yTop + yBottom) / 2);
      }
    };

    this.draw = function() {
      if (!this.loaded) {
        console.log('Data not yet loaded');
        return;
      }

      this.drawTitle();
      this.drawPaceZones();

      drawYAxisTickLabels(this.minPace,
                          this.maxPace,
                          this.layout,
                          this.mapPaceToHeight.bind(this),
                          1);

      drawAxis(this.layout);
      drawAxisLabels(this.xAxisLabel, this.yAxisLabel, this.layout);

      var dates = this.processedData.dates;
      var paces = this.processedData.paces;

      for (var i = 0; i < dates.length; i++) {
        var x = this.mapDateToWidth(dates[i]);
        var y = this.mapPaceToHeight(paces[i]);

        fill(100, 149, 237, 150);
        noStroke();
        ellipse(x, y, 6, 6);
      }

      var movingAvg = this.calculateMovingAverage(paces, 10);
      stroke(220, 20, 60);
      strokeWeight(2);
      noFill();
      beginShape();
      for (var j = 0; j < dates.length; j++) {
```

```
      var trendX = this.mapDateToWidth(dates[j]);
      var trendY = this.mapPaceToHeight(movingAvg[j]);
      vertex(trendX, trendY);
    }
    endShape();

    this.drawXAxisLabels();
    this.drawLegend();

    fill(100);
    noStroke();
    textSize(10);
    textAlign('left', 'top');
    text('Note: Lower pace = faster running',
         this.layout.leftMargin,
         this.layout.bottomMargin + 45);
  };

  this.drawTitle = function() {
    fill(0);
    noStroke();
    textAlign('center', 'center');
    textSize(18);
    text(this.title,
         (this.layout.plotWidth() / 2) + this.layout.leftMargin,
         this.layout.topMargin / 2);
  };

  this.drawXAxisLabels = function() {
    var dates = this.processedData.dates;
    var numLabels = Math.min(this.layout.numXTickLabels, dates.length);
    var step = Math.floor(dates.length / numLabels);

    fill(0);
    noStroke();
    textSize(10);
    textAlign('center', 'top');

    for (var i = 0; i < dates.length; i += step) {
      var x = this.mapDateToWidth(dates[i]);
      var dateLabel = this.formatDate(dates[i]);

      stroke(0);
      line(x, this.layout.bottomMargin, x, this.layout.bottomMargin + 5);

      noStroke();
      text(dateLabel, x, this.layout.bottomMargin + 8);
    }
  };

  this.formatDate = function(date) {
    var months = ['Jan', 'Feb', 'Mar', 'Apr', 'May', 'Jun',
                  'Jul', 'Aug', 'Sep', 'Oct', 'Nov', 'Dec'];
    return months[date.getMonth()] + ' ' + (date.getFullYear() % 100);
  };

  this.drawLegend = function() {
    var legendX = this.layout.rightMargin - 150;
    var legendY = this.layout.topMargin + 10;

    fill(100, 149, 237);
    noStroke();
    ellipse(legendX, legendY, 6, 6);
    fill(0);
    textAlign('left', 'center');
    textSize(11);
```

```
      text('Individual runs', legendX + 10, legendY);

      stroke(220, 20, 60);
      strokeWeight(2);
      line(legendX - 10, legendY + 20, legendX + 10, legendY + 20);
      noStroke();
      fill(0);
      text('10-run moving avg', legendX + 15, legendY + 20);
    };

    this.mapDateToWidth = function(date) {
      var startTime = this.startDate.getTime();
      var endTime = this.endDate.getTime();
      return map(date.getTime(),
                 startTime,
                 endTime,
                 this.layout.leftMargin,
                 this.layout.rightMargin);
    };

    this.mapPaceToHeight = function(value) {
      return map(value,
                 this.minPace,
                 this.maxPace,
                 this.layout.topMargin,
                 this.layout.bottomMargin);
    };
  }
  /* End of my own code */
```

9. activity-types.js

Activity Types — pie chart with dropdown switcher

```
/* Start of my own code */
// Activity Types: Pie chart showing distribution of Run, Trail Run, and
// Hiking activities. Uses a dropdown selector (createSelect) to switch
// between count-based and distance-based views. Delegates rendering to
// the template's PieChart constructor.
function ActivityTypes() {
  this.name = 'Activity Types';
  this.id = 'activity-types';
  this.loaded = false;

  this.pie = new PieChart(width / 2, height / 2, width * 0.35);

  this.preload = function() {
    var self = this;
    this.data = loadTable(
      './data/running/activities.csv', 'csv', 'header',
      function(table) {
        self.loaded = true;
      });
  };

  this.setup = function() {
    if (!this.loaded) {
      console.log('Data not yet loaded');
      return;
    }

    this.activityCounts = this.countActivityTypes();
    this.distanceByType = this.sumDistanceByType();

    this.select = createSelect();
    this.select.position(350, 40);
    this.select.option('By Count');
    this.select.option('By Distance');
  };

  this.destroy = function() {
    this.select.remove();
  };

  this.countActivityTypes = function() {
    var counts = {};

    for (var i = 0; i < this.data.getRowCount(); i++) {
      var activityType = this.data.getString(i, 'activity_type');

      if (counts[activityType]) {
        counts[activityType]++;
      } else {
        counts[activityType] = 1;
      }
    }

    return counts;
  };

  this.sumDistanceByType = function() {
    var distances = {};

    for (var i = 0; i < this.data.getRowCount(); i++) {
      var activityType = this.data.getString(i, 'activity_type');
```

```
      var distance = this.data.getNum(i, 'distance');

      if (distances[activityType]) {
        distances[activityType] += distance;
      } else {
        distances[activityType] = distance;
      }
    }

    return distances;
  };

  this.draw = function() {
    if (!this.loaded) {
      console.log('Data not yet loaded');
      return;
    }

    var viewMode = this.select.value();
    var dataToShow;
    var title;

    if (viewMode === 'By Count') {
      dataToShow = this.activityCounts;
      title = 'Activity Types by Count';
    } else {
      dataToShow = this.distanceByType;
      title = 'Activity Types by Distance (km)';
    }

    var labels = Object.keys(dataToShow);
    var values = [];
    for (var i = 0; i < labels.length; i++) {
      values.push(dataToShow[labels[i]]);
    }

    var colours = [
      color(66, 133, 244),
      color(52, 168, 83),
      color(251, 188, 5)
    ];

    this.pie.draw(values, labels, colours, title);
    this.drawStats(dataToShow, viewMode);
  };

  this.drawStats = function(data, viewMode) {
    var labels = Object.keys(data);
    var total = 0;
    for (var i = 0; i < labels.length; i++) {
      total += data[labels[i]];
    }

    var statsX = 80;
    var statsY = height - 100;

    fill(0);
    noStroke();
    textAlign('left', 'top');
    textSize(12);

    if (viewMode === 'By Count') {
      text('Total Activities: ' + total, statsX, statsY);
    } else {
      text('Total Distance: ' + total.toFixed(1) + ' km', statsX, statsY);
    }
```

```

    textSize(11);
    for (var j = 0; j < labels.length; j++) {
      var percentage = ((data[labels[j]] / total) * 100).toFixed(1);
      var valueStr = viewMode === 'By Count'
        ? data[labels[j]] + ' activities'
        : data[labels[j]].toFixed(1) + ' km';
      text(labels[j] + ': ' + valueStr + ' (' + percentage + '%)',
           statsX, statsY + 18 + (j * 16));
    }
  };
}
/* End of my own code */
```

10. heartrate-vs-pace.js

Heart Rate vs Pace — scatter plot with statistical analysis

```
/* Start of my own code */
// Heart Rate vs Pace: Scatter plot with statistical analysis showing the
// relationship between heart rate and running pace. Calculates Pearson's
// correlation coefficient and draws a least-squares linear regression line.
// Uses composition with ScatterPlot for point rendering and Tooltip for
// hover interaction.
function HeartRateVsPace() {
  this.name = 'Heart Rate vs Pace';
  this.id = 'heartrate-vs-pace';
  this.title = 'Heart Rate vs Running Pace';

  var marginSize = 35;
  this.layout = {
    marginSize: marginSize,
    leftMargin: marginSize * 2.5,
    rightMargin: width - marginSize,
    topMargin: marginSize * 2,
    bottomMargin: height - marginSize * 2.5,
    pad: 5,
    plotWidth: function() {
      return this.rightMargin - this.leftMargin;
    },
    plotHeight: function() {
      return this.bottomMargin - this.topMargin;
    },
    grid: true,
    numXTickLabels: 6,
    numYTickLabels: 6
  };

  this.scatterPlot = new ScatterPlot(this.layout);
  this.loaded = false;

  this.preload = function() {
    var self = this;
    this.data = loadTable(
      './data/running/activities.csv', 'csv', 'header',
      function(table) {
        self.loaded = true;
      });
  };

  this.setup = function() {
    if (!this.loaded) {
      console.log('Data not yet loaded');
      return;
    }

    this.processedData = this.processData();

    this.xRange = {
      min: Math.floor(min(this.processedData.heartRates) / 10) * 10 - 5,
      max: Math.ceil(max(this.processedData.heartRates) / 10) * 10 + 5
    };

    this.yRange = {
      min: Math.floor(min(this.processedData.paces) * 2) / 2 - 0.5,
      max: Math.ceil(max(this.processedData.paces) * 2) / 2 + 0.5
    };
  };

```

```
    this.destroy = function() {};

    this.processData = function() {
      var heartRates = [];
      var paces = [];
      var distances = [];
      var labels = [];

      for (var i = 0; i < this.data.getRowCount(); i++) {
        var activityType = this.data.getString(i, 'activity_type');

        if (activityType === 'Run') {
          var heartRate = this.data.getNum(i, 'avg_heart_rate');
          var paceStr = this.data.getString(i, 'avg_pace');
          var distance = this.data.getNum(i, 'distance');
          var date = this.data.getString(i, 'date');

          if (heartRate > 0 && distance >= 3) {
            var paceMinutes = paceToMinutes(paceStr);

            if (paceMinutes >= 5 && paceMinutes <= 9) {
              heartRates.push(heartRate);
              paces.push(paceMinutes);
              distances.push(distance);
              labels.push(date + ' - ' + distance.toFixed(1) + 'km');
            }
          }
        }
      }

      return {
        heartRates: heartRates,
        paces: paces,
        distances: distances,
        labels: labels
      };
    };

    // Pearson correlation coefficient: measures linear relationship between two variables.
    // Single-pass algorithm: O(n) time complexity where n = number of data points.
    this.calculateCorrelation = function(x, y) {
      var n = x.length;
      var sumX = 0, sumY = 0, sumXY = 0, sumX2 = 0, sumY2 = 0;

      for (var i = 0; i < n; i++) {
        sumX += x[i];
        sumY += y[i];
        sumXY += x[i] * y[i];
        sumX2 += x[i] * x[i];
        sumY2 += y[i] * y[i];
      }

      var numerator = (n * sumXY) - (sumX * sumY);
      var denominator = Math.sqrt(((n * sumX2) - (sumX * sumX)) *
                                  ((n * sumY2) - (sumY * sumY)));

      if (denominator === 0) return 0;
      return numerator / denominator;
    };

    // Least squares linear regression: finds best fit line y = mx + b.
    // O(n) time complexity — single pass through the data.
    this.calculateRegression = function(x, y) {
      var n = x.length;
      var sumX = 0, sumY = 0, sumXY = 0, sumX2 = 0;

```

```
    for (var i = 0; i < n; i++) {
      sumX += x[i];
      sumY += y[i];
      sumXY += x[i] * y[i];
      sumX2 += x[i] * x[i];
    }

    var slope = ((n * sumXY) - (sumX * sumY)) / ((n * sumX2) - (sumX * sumX));
    var intercept = (sumY - (slope * sumX)) / n;

    return { slope: slope, intercept: intercept };
  };

  this.draw = function() {
    if (!this.loaded) {
      console.log('Data not yet loaded');
      return;
    }

    fill(0);
    noStroke();
    textAlign('center', 'center');
    textSize(18);
    text(this.title,
         (this.layout.leftMargin + this.layout.rightMargin) / 2,
         this.layout.topMargin / 2);

    stroke(0);
    strokeWeight(1);
    line(this.layout.leftMargin, this.layout.topMargin,
         this.layout.leftMargin, this.layout.bottomMargin);
    line(this.layout.leftMargin, this.layout.bottomMargin,
         this.layout.rightMargin, this.layout.bottomMargin);

    this.scatterPlot.drawXAxisTicks(this.xRange.min, this.xRange.max,
                                    this.layout.numXTickLabels,
                                    'Average Heart Rate (bpm)');
    this.scatterPlot.drawYAxisTicks(this.yRange.min, this.yRange.max,
                                    this.layout.numYTickLabels,
                                    'Pace (min/km)');

    var regression = this.calculateRegression(
      this.processedData.heartRates,
      this.processedData.paces
    );
    this.drawTrendLine(regression);

    // Map distance to point size: longer runs get larger bubbles
    var distances = this.processedData.distances;
    var minDist = min(distances);
    var maxDist = max(distances);
    var sizes = [];
    for (var i = 0; i < distances.length; i++) {
      sizes.push(map(distances[i], minDist, maxDist, 4, 28));
    }

    var pointColour = color(66, 133, 244, 180);
    this.scatterPlot.draw(
      this.processedData.heartRates,
      this.processedData.paces,
      this.xRange,
      this.yRange,
      pointColour,
      this.processedData.labels,
      sizes
    );
```

```

      this.drawCorrelationInfo();

      fill(100);
      textSize(10);
      textAlign('left', 'top');
      text('Hover over points to see details',
           this.layout.leftMargin,
           this.layout.bottomMargin + 50);
    };

    this.drawTrendLine = function(regression) {
      var x1 = this.xRange.min;
      var x2 = this.xRange.max;
      var y1 = regression.slope * x1 + regression.intercept;
      var y2 = regression.slope * x2 + regression.intercept;

      y1 = constrain(y1, this.yRange.min, this.yRange.max);
      y2 = constrain(y2, this.yRange.min, this.yRange.max);

      var screenX1 = this.scatterPlot.mapToX(x1, this.xRange.min, this.xRange.max);
      var screenY1 = this.scatterPlot.mapToY(y1, this.yRange.min, this.yRange.max);
      var screenX2 = this.scatterPlot.mapToX(x2, this.xRange.min, this.xRange.max);
      var screenY2 = this.scatterPlot.mapToY(y2, this.yRange.min, this.yRange.max);

      stroke(220, 20, 60);
      strokeWeight(2);
      line(screenX1, screenY1, screenX2, screenY2);
    };

    this.drawCorrelationInfo = function() {
      var correlation = this.calculateCorrelation(
        this.processedData.heartRates,
        this.processedData.paces
      );

      var infoX = this.layout.rightMargin - 180;
      var infoY = this.layout.topMargin + 10;

      fill(255, 255, 255, 200);
      stroke(200);
      strokeWeight(1);
      rect(infoX - 10, infoY - 5, 180, 70, 5);

      fill(0);
      noStroke();
      textAlign('left', 'top');
      textSize(11);

      text('Correlation: ' + correlation.toFixed(3), infoX, infoY);
      text('Data points: ' + this.processedData.heartRates.length, infoX, infoY + 16);

      textSize(10);
      fill(100);
      var interpretation;
      if (correlation < -0.3) {
        interpretation = 'Negative correlation: higher HR,';
        text(interpretation, infoX, infoY + 36);
        text('faster pace (lower number)', infoX, infoY + 48);
      } else if (correlation > 0.3) {
        interpretation = 'Positive correlation: higher HR,';
        text(interpretation, infoX, infoY + 36);
        text('slower pace', infoX, infoY + 48);
      } else {
        text('Weak correlation', infoX, infoY + 36);
      }
```

```
  };
}
/* End of my own code */
```

11. goal-tracker.js

Goal Tracker — semicircular gauge with trigonometry

```
/* Start of my own code */
// Goal Tracker: Semicircular gauge displaying monthly distance progress
// toward a 100 km goal. Uses trigonometric functions (cos/sin) for polar
// coordinate positioning along a 270-degree arc, with a five-tier colour
// system (red → green) for visual feedback.
function GoalTracker() {
  this.name = 'Goal Tracker';
  this.id = 'goal-tracker';
  this.title = 'Monthly Distance Goal';
  this.loaded = false;

  // Goal: 100km per month
  this.monthlyGoal = 100;

  // Gauge settings
  this.gaugeX = width / 2;
  this.gaugeY = height / 2 - 30;
  this.gaugeRadius = 150;
  this.gaugeWeight = 25;

  this.preload = function() {
    var self = this;
    this.data = loadTable(
      './data/running/activities.csv', 'csv', 'header',
      function(table) {
        self.loaded = true;
      });
  };

  this.setup = function() {
    if (!this.loaded) {
      console.log('Data not yet loaded');
      return;
    }

    this.monthlyData = this.calculateMonthlyDistances();
    this.currentMonth = this.getCurrentMonthKey();

    // Create month selector
    this.select = createSelect();
    this.select.position(420, 40);

    var months = Object.keys(this.monthlyData).sort().reverse();
    for (var i = 0; i < months.length; i++) {
      this.select.option(this.formatMonthLabel(months[i]), months[i]);
    }
    this.select.selected(this.currentMonth);
  };

  this.destroy = function() {
    this.select.remove();
  };

  this.getCurrentMonthKey = function() {
    var now = new Date();
    var year = now.getFullYear();
    var month = (now.getMonth() + 1).toString().padStart(2, '0');
    return year + '-' + month;
  };

  this.formatMonthLabel = function(yearMonth) {
```

```
    var parts = yearMonth.split('-');
    var monthNames = ['Jan', 'Feb', 'Mar', 'Apr', 'May', 'Jun',
                      'Jul', 'Aug', 'Sep', 'Oct', 'Nov', 'Dec'];
    return monthNames[parseInt(parts[1]) - 1] + ' ' + parts[0];
  };

  this.calculateMonthlyDistances = function() {
    var months = {};

    for (var i = 0; i < this.data.getRowCount(); i++) {
      var dateStr = this.data.getString(i, 'date');
      var distance = this.data.getNum(i, 'distance');
      var yearMonth = dateStr.substring(0, 7);

      if (months[yearMonth]) {
        months[yearMonth].total += distance;
        months[yearMonth].runs.push({
          date: dateStr,
          distance: distance
        });
      } else {
        months[yearMonth] = {
          total: distance,
          runs: [{
            date: dateStr,
            distance: distance
          }]
        };
      }
    }

    return months;
  };

  this.draw = function() {
    if (!this.loaded) {
      console.log('Data not yet loaded');
      return;
    }

    var selectedMonth = this.select.value();
    var monthData = this.monthlyData[selectedMonth];

    if (!monthData) {
      fill(0);
      textAlign('center', 'center');
      textSize(16);
      text('No data for selected month', width / 2, height / 2);
      return;
    }

    var currentDistance = monthData.total;
    var progress = currentDistance / this.monthlyGoal;
    var progressPercent = Math.min(progress * 100, 100);

    // Draw title
    fill(0);
    noStroke();
    textAlign('center', 'center');
    textSize(20);
    text(this.title, width / 2, 40);

    textSize(14);
    fill(100);
    text('Target: ' + this.monthlyGoal + ' km/month', width / 2, 65);

```

```
      // Draw gauge
      this.drawGauge(progressPercent, currentDistance);

      // Draw stats
      this.drawStats(monthData, currentDistance, progress);

      // Draw recent runs
      this.drawRecentRuns(monthData.runs);
    };

    this.drawGauge = function(percent, distance) {
      var startAngle = PI * 0.75;
      var endAngle = PI * 2.25;
      var totalArc = endAngle - startAngle;

      // Background arc
      noFill();
      stroke(230);
      strokeWeight(this.gaugeWeight);
      strokeCap(ROUND);
      arc(this.gaugeX, this.gaugeY, this.gaugeRadius * 2, this.gaugeRadius * 2,
          startAngle, endAngle);

      // Progress arc with color based on progress
      var progressAngle = startAngle + (totalArc * Math.min(percent / 100, 1));
      var gaugeColor = this.getProgressColor(percent);
      stroke(gaugeColor);
      strokeWeight(this.gaugeWeight);
      arc(this.gaugeX, this.gaugeY, this.gaugeRadius * 2, this.gaugeRadius * 2,
          startAngle, progressAngle);

      // Center text
      noStroke();
      fill(0);
      textAlign('center', 'center');
      textSize(42);
      textStyle(BOLD);
      text(distance.toFixed(1), this.gaugeX, this.gaugeY - 10);

      textSize(16);
      textStyle(NORMAL);
      fill(100);
      text('km', this.gaugeX, this.gaugeY + 20);

      // Percentage
      textSize(18);
      fill(gaugeColor);
      text(percent.toFixed(0) + '%', this.gaugeX, this.gaugeY + 50);

      // Goal markers
      this.drawGoalMarkers(startAngle, totalArc);
    };

    this.drawGoalMarkers = function(startAngle, totalArc) {
      var markers = [0, 25, 50, 75, 100];
      var markerRadius = this.gaugeRadius + 25;

      fill(100);
      noStroke();
      textSize(10);
      textAlign('center', 'center');

      for (var i = 0; i < markers.length; i++) {
        var angle = startAngle + (totalArc * markers[i] / 100);
        var x = this.gaugeX + cos(angle) * markerRadius;
        var y = this.gaugeY + sin(angle) * markerRadius;
```

```
      text(markers[i] + 'km', x, y);
    }
  };

  this.getProgressColor = function(percent) {
    if (percent >= 100) {
      return color(46, 204, 113); // Green - goal achieved
    } else if (percent >= 75) {
      return color(52, 152, 219); // Blue - on track
    } else if (percent >= 50) {
      return color(241, 196, 15); // Yellow - halfway
    } else if (percent >= 25) {
      return color(230, 126, 34); // Orange - needs effort
    } else {
      return color(231, 76, 60); // Red - just started
    }
  };

  this.drawStats = function(monthData, currentDistance, progress) {
    var statsX = 80;
    var statsY = height - 140;

    fill(0);
    noStroke();
    textAlign('left', 'top');
    textSize(14);
    textStyle(BOLD);
    text('Statistics', statsX, statsY);

    textSize(12);
    textStyle(NORMAL);
    fill(60);

    var remaining = Math.max(0, this.monthlyGoal - currentDistance);
    var numRuns = monthData.runs.length;
    var avgPerRun = numRuns > 0 ? currentDistance / numRuns : 0;

    text('Total runs: ' + numRuns, statsX, statsY + 25);
    text('Average per run: ' + avgPerRun.toFixed(1) + ' km', statsX, statsY + 45);
    text('Remaining: ' + remaining.toFixed(1) + ' km', statsX, statsY + 65);

    if (progress >= 1) {
      fill(46, 204, 113);
      textStyle(BOLD);
      text('Goal achieved!', statsX, statsY + 90);
    } else {
      fill(100);
      var runsNeeded = Math.ceil(remaining / avgPerRun);
      if (avgPerRun > 0 && runsNeeded > 0) {
        text('~' + runsNeeded + ' more runs needed (at current avg)', statsX, statsY + 90);
      }
    }
  };

  this.drawRecentRuns = function(runs) {
    var listX = width - 250;
    var listY = height - 140;

    fill(0);
    noStroke();
    textAlign('left', 'top');
    textSize(14);
    textStyle(BOLD);
    text('Recent Runs', listX, listY);

    textSize(11);
```

```
      textStyle(NORMAL);
      fill(60);

      // Sort by date descending and show last 5
      var sortedRuns = runs.slice().sort(function(a, b) {
        return new Date(b.date) - new Date(a.date);
      });

      var displayCount = Math.min(5, sortedRuns.length);
      for (var i = 0; i < displayCount; i++) {
        var run = sortedRuns[i];
        var dateFormatted = this.formatDate(run.date);
        text(dateFormatted + ': ' + run.distance.toFixed(1) + ' km',
             listX, listY + 25 + (i * 18));
      }
    };

    this.formatDate = function(dateStr) {
      var date = new Date(dateStr);
      var months = ['Jan', 'Feb', 'Mar', 'Apr', 'May', 'Jun',
                    'Jul', 'Aug', 'Sep', 'Oct', 'Nov', 'Dec'];
      return months[date.getMonth()] + ' ' + date.getDate();
    };
  }
  /* End of my own code */
```

12. training-heatmap.js

Training Heatmap — GitHub-style calendar grid

```
/* Start of my own code */
// Training Heatmap: GitHub-style contribution calendar for running data.
// Displays daily running distances on a weekly calendar grid using
// nested loops (weeks x weekdays) and distance-to-colour mapping.
function TrainingHeatmap() {
  this.name = 'Training Heatmap';
  this.id = 'training-heatmap';
  this.title = 'Training Consistency Heatmap';
  this.loaded = false;

  // Grid layout settings
  this.cellSize = 13;
  this.cellGap = 2;
  this.startX = 105;
  this.startY = 110;

  // GitHub-inspired green colour scale
  this.colors = [
    [235, 237, 240],  // Level 0: No activity (light gray)
    [155, 233, 168],  // Level 1: Short distance (light green)
    [64, 196, 99],    // Level 2: Medium distance
    [48, 161, 78],    // Level 3: High distance
    [33, 110, 57]     // Level 4: Very high distance (dark green)
  ];

  this.tooltip = new Tooltip();

  this.preload = function() {
    var self = this;
    this.data = loadTable(
      './data/running/activities.csv', 'csv', 'header',
      function(table) {
        self.loaded = true;
      });
  };

  this.setup = function() {
    if (!this.loaded) {
      console.log('Data not yet loaded');
      return;
    }

    this.dailyData = this.processDailyData();
    this.years = Object.keys(this.dailyData).sort().reverse();

    // Create year selector dropdown
    this.select = createSelect();
    this.select.position(420, 40);
    for (var i = 0; i < this.years.length; i++) {
      this.select.option(this.years[i]);
    }
  };

  this.destroy = function() {
    this.select.remove();
  };

  // Aggregate daily distances grouped by year and date
  this.processDailyData = function() {
    var data = {};

```

```
    for (var i = 0; i < this.data.getRowCount(); i++) {
      var dateStr = this.data.getString(i, 'date');
      var distance = this.data.getNum(i, 'distance');
      var year = dateStr.substring(0, 4);

      if (!data[year]) {
        data[year] = {};
      }

      if (data[year][dateStr]) {
        data[year][dateStr] += distance;
      } else {
        data[year][dateStr] = distance;
      }
    }

    return data;
  };

  this.draw = function() {
    if (!this.loaded) {
      console.log('Data not yet loaded');
      return;
    }

    var selectedYear = this.select.value();
    var yearData = this.dailyData[selectedYear] || {};

    // Title
    fill(0);
    noStroke();
    textAlign('center', 'center');
    textSize(20);
    text(this.title, width / 2, 40);

    textSize(14);
    fill(100);
    text(selectedYear, width / 2, 65);

    // Draw calendar components
    this.drawWeekdayLabels();
    this.drawMonthLabels(parseInt(selectedYear));
    var hoveredCell = this.drawCalendarGrid(parseInt(selectedYear), yearData);
    this.drawLegend();
    this.drawYearStats(yearData);

    // Draw tooltip last so it appears on top of all cells
    if (hoveredCell) {
      this.tooltip.draw(mouseX, mouseY, hoveredCell.lines, null);
    }
  };

  // Draw weekday labels (Mon, Wed, Fri only to avoid crowding)
  this.drawWeekdayLabels = function() {
    var days = ['Sun', 'Mon', 'Tue', 'Wed', 'Thu', 'Fri', 'Sat'];
    fill(100);
    noStroke();
    textSize(9);
    textAlign('right', 'center');

    for (var d = 0; d < 7; d++) {
      if (d == 1 || d == 3 || d == 5) {
        var y = this.startY + d * (this.cellSize + this.cellGap) + this.cellSize / 2;
        text(days[d], this.startX - 8, y);
      }
    }
```

```
    };

    // Draw month labels along the top of the grid
    this.drawMonthLabels = function(year) {
      var months = ['Jan', 'Feb', 'Mar', 'Apr', 'May', 'Jun',
                    'Jul', 'Aug', 'Sep', 'Oct', 'Nov', 'Dec'];
      var jan1 = new Date(year, 0, 1);
      var startDayOfWeek = jan1.getDay();

      fill(100);
      noStroke();
      textSize(9);
      textAlign('left', 'bottom');

      for (var m = 0; m < 12; m++) {
        var firstOfMonth = new Date(year, m, 1);
        var dayOfYear = Math.floor((firstOfMonth - jan1) / (24 * 60 * 60 * 1000));
        var weekNum = Math.floor((dayOfYear + startDayOfWeek) / 7);

        var x = this.startX + weekNum * (this.cellSize + this.cellGap);
        text(months[m], x, this.startY - 5);
      }
    };

    // Draw the calendar grid using nested loops (weeks x weekdays).
    // O(W * 7) where W = number of weeks (~53), so O(365) per frame.
    // Returns hovered cell info for tooltip rendering.
    this.drawCalendarGrid = function(year, yearData) {
      var jan1 = new Date(year, 0, 1);
      var startDayOfWeek = jan1.getDay();

      // Calculate max distance for colour scaling
      var maxDist = 0;
      var allDistances = Object.values(yearData);
      for (var d = 0; d < allDistances.length; d++) {
        if (allDistances[d] > maxDist) {
          maxDist = allDistances[d];
        }
      }
      if (maxDist === 0) {
        maxDist = 1;
      }

      var hoveredCell = null;
      var isLeapYear = (year % 4 === 0 && year % 100 !== 0) || (year % 400 === 0);
      var totalDays = isLeapYear ? 366 : 365;

      // Nested loop: iterate through each week and each day within that week
      var totalWeeks = Math.ceil((totalDays + startDayOfWeek) / 7);

      for (var week = 0; week < totalWeeks; week++) {
        for (var dayOfWeek = 0; dayOfWeek < 7; dayOfWeek++) {
          var dayIndex = week * 7 + dayOfWeek - startDayOfWeek;

          // Skip cells before Jan 1 and after Dec 31
          if (dayIndex < 0 || dayIndex >= totalDays) {
            continue;
          }

          var cellDate = new Date(year, 0, 1 + dayIndex);
          var dateStr = this.formatDateStr(cellDate);
          var distance = yearData[dateStr] || 0;

          var x = this.startX + week * (this.cellSize + this.cellGap);
          var y = this.startY + dayOfWeek * (this.cellSize + this.cellGap);

```

```
        // Map distance to colour
        var c = this.getDistanceColor(distance, maxDist);
        fill(c[0], c[1], c[2]);
        noStroke();
        rect(x, y, this.cellSize, this.cellSize, 2);

        // Check if mouse hovers over this cell
        if (mouseX >= x && mouseX <= x + this.cellSize &&
            mouseY >= y && mouseY <= y + this.cellSize) {
          // Highlight hovered cell
          noFill();
          stroke(0);
          strokeWeight(1);
          rect(x, y, this.cellSize, this.cellSize, 2);

          var lines = [dateStr];
          if (distance > 0) {
            lines.push(distance.toFixed(1) + ' km');
          } else {
            lines.push('Rest day');
          }
          hoveredCell = { lines: lines };
        }
      }
    }

    return hoveredCell;
  };

  // Format a Date object to "YYYY-MM-DD" string
  this.formatDateStr = function(date) {
    var y = date.getFullYear();
    var m = (date.getMonth() + 1).toString().padStart(2, '0');
    var d = date.getDate().toString().padStart(2, '0');
    return y + '-' + m + '-' + d;
  };

  // Map distance to colour level using quartile-based thresholds
  this.getDistanceColor = function(distance, maxDist) {
    if (distance === 0) {
      return this.colors[0];
    }

    var ratio = distance / maxDist;
    if (ratio <= 0.25) return this.colors[1];
    if (ratio <= 0.50) return this.colors[2];
    if (ratio <= 0.75) return this.colors[3];
    return this.colors[4];
  };

  // Draw colour legend (Less → More)
  this.drawLegend = function() {
    var legendX = width - 220;
    var legendY = this.startY + 7 * (this.cellSize + this.cellGap) + 30;

    fill(100);
    noStroke();
    textSize(10);
    textAlign('right', 'center');
    text('Less', legendX - 5, legendY + this.cellSize / 2);

    for (var i = 0; i < this.colors.length; i++) {
      var c = this.colors[i];
      fill(c[0], c[1], c[2]);
      noStroke();
      rect(legendX + i * (this.cellSize + 2), legendY, this.cellSize, this.cellSize, 2);
```

```
      }

      fill(100);
      textAlign('left', 'center');
      text('More', legendX + this.colors.length * (this.cellSize + 2) + 5,
           legendY + this.cellSize / 2);
    };

    // Draw year summary statistics below the grid
    this.drawYearStats = function(yearData) {
      var totalDist = 0;
      var activeDays = 0;
      var maxSingleRun = 0;
      var allDistances = Object.values(yearData);

      for (var i = 0; i < allDistances.length; i++) {
        totalDist += allDistances[i];
        activeDays++;
        if (allDistances[i] > maxSingleRun) {
          maxSingleRun = allDistances[i];
        }
      }

      var statsX = this.startX;
      var statsY = this.startY + 7 * (this.cellSize + this.cellGap) + 25;

      fill(0);
      noStroke();
      textSize(13);
      textAlign('left', 'top');
      textStyle(BOLD);
      text('Year Summary', statsX, statsY);

      textStyle(NORMAL);
      textSize(12);
      fill(60);
      text('Total distance: ' + totalDist.toFixed(1) + ' km', statsX, statsY + 22);
      text('Active days: ' + activeDays, statsX, statsY + 40);
      text('Average per active day: ' +
           (activeDays > 0 ? (totalDist / activeDays).toFixed(1) : '0') + ' km',
           statsX, statsY + 58);
      text('Longest run: ' + maxSingleRun.toFixed(1) + ' km', statsX, statsY + 76);
    };
  }
  /* End of my own code */
```

13. marathon-radar.js

Marathon Readiness Radar — spider chart with 5 normalised metrics

```
/* Start of my own code */
// Marathon Readiness Radar: spider chart plotting five normalised training
// metrics using trigonometry (sin/cos at 72-degree intervals).
// Provides a multi-dimensional readiness assessment in a single view.
function MarathonRadar() {
  this.name = 'Marathon Readiness';
  this.id = 'marathon-radar';
  this.title = 'Marathon Readiness Radar';
  this.loaded = false;

  // Radar chart settings
  this.centerX = width / 2;
  this.centerY = height / 2 + 10;
  this.radius = 170;
  this.numAxes = 5;
  // 72 degrees between each axis (360 / 5 = 72)
  this.angleStep = TWO_PI / this.numAxes;
  // Start from top (-PI/2) so first axis points upward
  this.startAngle = -PI / 2;

  // Five training metrics with labels and targets
  this.metrics = [
    { label: 'Monthly\nDistance', unit: 'km', target: 100 },
    { label: 'Average\nPace', unit: 'min/km', target: 6.0 },
    { label: 'Training\nConsistency', unit: 'days', target: 30 },
    { label: 'Long Runs\n(10km+)', unit: 'runs', target: 4 },
    { label: 'Heart Rate\nEfficiency', unit: 'bpm', target: 140 }
  ];

  this.tooltip = new Tooltip();

  this.preload = function() {
    var self = this;
    this.data = loadTable(
      './data/running/activities.csv', 'csv', 'header',
      function(table) {
        self.loaded = true;
      });
  };

  this.setup = function() {
    if (!this.loaded) {
      console.log('Data not yet loaded');
      return;
    }

    this.monthlyMetrics = this.calculateAllMonthlyMetrics();
    this.months = Object.keys(this.monthlyMetrics).sort().reverse();

    // Create month selector
    this.select = createSelect();
    this.select.position(420, 40);

    for (var i = 0; i < this.months.length; i++) {
      this.select.option(this.formatMonthLabel(this.months[i]), this.months[i]);
    }
  };

  this.destroy = function() {
    this.select.remove();
  };
```

```

    this.formatMonthLabel = function(yearMonth) {
      var parts = yearMonth.split('-');
      var monthNames = ['Jan', 'Feb', 'Mar', 'Apr', 'May', 'Jun',
                        'Jul', 'Aug', 'Sep', 'Oct', 'Nov', 'Dec'];
      return monthNames[parseInt(parts[1]) - 1] + ' ' + parts[0];
    };

    // Calculate all five metrics for each month
    this.calculateAllMonthlyMetrics = function() {
      var months = {};

      for (var i = 0; i < this.data.getRowCount(); i++) {
        var activityType = this.data.getString(i, 'activity_type');
        if (activityType !== 'Run') continue;

        var dateStr = this.data.getString(i, 'date');
        var distance = this.data.getNum(i, 'distance');
        var paceStr = this.data.getString(i, 'avg_pace');
        var heartRate = this.data.getNum(i, 'avg_heart_rate');
        var yearMonth = dateStr.substring(0, 7);

        if (!months[yearMonth]) {
          months[yearMonth] = {
            totalDistance: 0,
            paces: [],
            activeDays: {},
            longRuns: 0,
            heartRates: []
          };
        }

        months[yearMonth].totalDistance += distance;

        var pace = paceToMinutes(paceStr);
        if (pace > 0) {
          months[yearMonth].paces.push(pace);
        }

        months[yearMonth].activeDays[dateStr] = true;

        if (distance >= 10) {
          months[yearMonth].longRuns++;
        }

        if (heartRate > 0) {
          months[yearMonth].heartRates.push(heartRate);
        }
      }

      // Normalise each metric to 0-1 scale
      var result = {};
      var monthKeys = Object.keys(months);

      for (var m = 0; m < monthKeys.length; m++) {
        var key = monthKeys[m];
        var raw = months[key];

        var avgPace = 0;
        if (raw.paces.length > 0) {
          var paceSum = 0;
          for (var p = 0; p < raw.paces.length; p++) {
            paceSum += raw.paces[p];
          }
          avgPace = paceSum / raw.paces.length;
        }
```

```

      var avgHR = 0;
      if (raw.heartRates.length > 0) {
        var hrSum = 0;
        for (var h = 0; h < raw.heartRates.length; h++) {
          hrSum += raw.heartRates[h];
        }
        avgHR = hrSum / raw.heartRates.length;
      }

      var activeDayCount = Object.keys(raw.activeDays).length;

      result[key] = {
        raw: [
          raw.totalDistance,
          avgPace,
          activeDayCount,
          raw.longRuns,
          avgHR
        ],
        normalised: [
          // 1. Monthly distance: 0-100km → 0-1 (capped at 1)
          Math.min(raw.totalDistance / 100, 1),
          // 2. Average pace: 6:00 = perfect (1.0), 9:00 = worst (0.0)
          avgPace > 0 ? Math.max(0, Math.min(1, 1 - (avgPace - 6.0) / 3.0)) : 0,
          // 3. Consistency: active days / 30
          Math.min(activeDayCount / 30, 1),
          // 4. Long runs: count / target of 4
          Math.min(raw.longRuns / 4, 1),
          // 5. Heart rate efficiency: lower avg HR = better
          // 130 bpm = perfect (1.0), 170 bpm = worst (0.0)
          avgHR > 0 ? Math.max(0, Math.min(1, 1 - (avgHR - 130) / 40)) : 0
        ]
      };
    }

    return result;
  };

  this.draw = function() {
    if (!this.loaded) {
      console.log('Data not yet loaded');
      return;
    }

    var selectedMonth = this.select.value();
    var monthData = this.monthlyMetrics[selectedMonth];

    // Title
    fill(0);
    noStroke();
    textAlign('center', 'center');
    textSize(20);
    text(this.title, width / 2, 40);

    textSize(14);
    fill(100);
    text(this.formatMonthLabel(selectedMonth), width / 2, 65);

    if (!monthData) {
      textSize(16);
      text('No running data for selected month', width / 2, height / 2);
      return;
    }

    // Draw radar chart components
```

```
      this.drawGridLines();
      this.drawAxes();
      this.drawDataPolygon(monthData.normalised);
      this.drawAxisLabels(monthData);
      this.drawScorecard(monthData);
    };

    // Draw concentric pentagons as reference grid (20%, 40%, 60%, 80%, 100%)
    this.drawGridLines = function() {
      var levels = [0.2, 0.4, 0.6, 0.8, 1.0];

      for (var lvl = 0; lvl < levels.length; lvl++) {
        var r = this.radius * levels[lvl];

        stroke(220);
        strokeWeight(0.5);
        noFill();
        beginShape();

        for (var i = 0; i < this.numAxes; i++) {
          var angle = this.startAngle + i * this.angleStep;
          var x = this.centerX + cos(angle) * r;
          var y = this.centerY + sin(angle) * r;
          vertex(x, y);
        }

        endShape(CLOSE);

        // Percentage label on first axis
        if (lvl < levels.length) {
          fill(180);
          noStroke();
          textSize(8);
          textAlign('center', 'bottom');
          var labelY = this.centerY - r - 2;
          text(Math.round(levels[lvl] * 100) + '%', this.centerX, labelY);
        }
      }
    };

    // Draw the five axes from center to edge
    this.drawAxes = function() {
      stroke(200);
      strokeWeight(1);

      for (var i = 0; i < this.numAxes; i++) {
        var angle = this.startAngle + i * this.angleStep;
        var x = this.centerX + cos(angle) * this.radius;
        var y = this.centerY + sin(angle) * this.radius;
        line(this.centerX, this.centerY, x, y);
      }
    };

    // Draw the filled data polygon connecting all five metric values
    this.drawDataPolygon = function(values) {
      // Filled polygon
      fill(66, 133, 244, 80);
      stroke(66, 133, 244);
      strokeWeight(2);
      beginShape();

      for (var i = 0; i < this.numAxes; i++) {
        var angle = this.startAngle + i * this.angleStep;
        var r = this.radius * values[i];
        var x = this.centerX + cos(angle) * r;
        var y = this.centerY + sin(angle) * r;
```

```
        vertex(x, y);
      }

      endShape(CLOSE);

      // Data points at each vertex
      for (var j = 0; j < this.numAxes; j++) {
        var pointAngle = this.startAngle + j * this.angleStep;
        var pointR = this.radius * values[j];
        var px = this.centerX + cos(pointAngle) * pointR;
        var py = this.centerY + sin(pointAngle) * pointR;

        fill(66, 133, 244);
        noStroke();
        ellipse(px, py, 8, 8);
      }
    };

    // Draw metric labels at the end of each axis
    this.drawAxisLabels = function(monthData) {
      var labelOffset = 30;

      for (var i = 0; i < this.numAxes; i++) {
        var angle = this.startAngle + i * this.angleStep;
        var x = this.centerX + cos(angle) * (this.radius + labelOffset);
        var y = this.centerY + sin(angle) * (this.radius + labelOffset);

        fill(0);
        noStroke();
        textSize(10);
        textStyle(BOLD);
        textAlign('center', 'center');

        // Split label into multiple lines
        var labelLines = this.metrics[i].label.split('\n');
        for (var l = 0; l < labelLines.length; l++) {
          text(labelLines[l], x, y + (l * 13) - 6);
        }

        // Show raw value below label
        textStyle(NORMAL);
        textSize(9);
        fill(100);
        var rawVal = monthData.raw[i];
        var displayVal;

        if (i === 0) {
          displayVal = rawVal.toFixed(1) + ' km';
        } else if (i === 1) {
          displayVal = formatPace(rawVal) + '/km';
        } else if (i === 2) {
          displayVal = rawVal + ' days';
        } else if (i === 3) {
          displayVal = rawVal + ' runs';
        } else {
          displayVal = rawVal > 0 ? rawVal.toFixed(0) + ' bpm' : 'N/A';
        }

        text(displayVal, x, y + labelLines.length * 13 - 2);
      }
    };

    // Draw overall readiness score and interpretation
    this.drawScorecard = function(monthData) {
      var values = monthData.normalised;
      var totalScore = 0;
```

```
    for (var i = 0; i < values.length; i++) {
      totalScore += values[i];
    }
    var avgScore = totalScore / values.length;
    var percent = Math.round(avgScore * 100);

    var cardX = 30;
    var cardY = height - 90;

    fill(0);
    noStroke();
    textAlign('left', 'top');
    textSize(14);
    textStyle(BOLD);
    text('Readiness Score', cardX, cardY);

    // Color-coded score
    if (percent >= 75) {
      fill(46, 204, 113);
    } else if (percent >= 50) {
      fill(52, 152, 219);
    } else if (percent >= 25) {
      fill(241, 196, 15);
    } else {
      fill(231, 76, 60);
    }
    textSize(28);
    text(percent + '%', cardX, cardY + 22);

    // Interpretation
    textStyle(NORMAL);
    textSize(11);
    fill(100);
    var interpretation;
    if (percent >= 75) {
      interpretation = 'Excellent - Ready for marathon';
    } else if (percent >= 50) {
      interpretation = 'Good - On track, keep training';
    } else if (percent >= 25) {
      interpretation = 'Fair - More training needed';
    } else {
      interpretation = 'Low - Significant work needed';
    }
    text(interpretation, cardX, cardY + 60);
  };
}
/* End of my own code */
```

14. tests.js

Unit and integration tests (TestRunner, black-box testing)

```
/* Start of my own code */
// Unit and integration tests for the Running Data Visualisation App.
// Implements concepts from Week 17: unit testing, black-box testing,
// and test-driven validation of data processing functions.

function TestRunner() {
  this.passed = 0;
  this.failed = 0;
  this.results = [];

  // Core assertion: compares actual vs expected values
  this.assertEquals = function(testName, actual, expected) {
    if (actual === expected) {
      this.passed++;
      this.results.push({ name: testName, status: 'PASS', detail: '' });
    } else {
      this.failed++;
      this.results.push({
        name: testName,
        status: 'FAIL',
        detail: 'Expected ' + expected + ', got ' + actual
      });
    }
  };

  // Assertion for floating point comparison with tolerance
  this.assertAlmostEqual = function(testName, actual, expected, tolerance) {
    if (Math.abs(actual - expected) <= tolerance) {
      this.passed++;
      this.results.push({ name: testName, status: 'PASS', detail: '' });
    } else {
      this.failed++;
      this.results.push({
        name: testName,
        status: 'FAIL',
        detail: 'Expected ~' + expected + ', got ' + actual
      });
    }
  };

  // Assertion for boolean true
  this.assertTrue = function(testName, actual) {
    this.assertEquals(testName, actual, true);
  };

  // Display results on the p5.js canvas
  this.displayResults = function() {
    background(255);
    var y = 30;

    fill(0);
    noStroke();
    textSize(18);
    textStyle(BOLD);
    textAlign('left', 'top');
    text('Test Results', 20, y);
    y += 30;

    // Summary
    textSize(14);
    textStyle(NORMAL);
```

```
    var total = this.passed + this.failed;
    if (this.failed === 0) {
      fill(46, 204, 113);
      text('All ' + total + ' tests passed', 20, y);
    } else {
      fill(231, 76, 60);
      text(this.failed + ' of ' + total + ' tests failed', 20, y);
    }
    y += 30;

    // Individual results
    textSize(11);
    for (var i = 0; i < this.results.length; i++) {
      var r = this.results[i];
      if (r.status === 'PASS') {
        fill(46, 204, 113);
        text('[PASS] ' + r.name, 20, y);
      } else {
        fill(231, 76, 60);
        text('[FAIL] ' + r.name + ' - ' + r.detail, 20, y);
      }
      y += 16;
    }
  };
}

// ==============================================================
// Unit Tests: helper-functions.js
// ==============================================================

function testPaceToMinutes(runner) {
  // Black-box tests: valid inputs
  runner.assertAlmostEqual(
    'paceToMinutes("6:30") should return 6.5',
    paceToMinutes('6:30'), 6.5, 0.01
  );
  runner.assertAlmostEqual(
    'paceToMinutes("5:00") should return 5.0',
    paceToMinutes('5:00'), 5.0, 0.01
  );
  runner.assertAlmostEqual(
    'paceToMinutes("7:15") should return 7.25',
    paceToMinutes('7:15'), 7.25, 0.01
  );
  runner.assertAlmostEqual(
    'paceToMinutes("6:00") should return 6.0',
    paceToMinutes('6:00'), 6.0, 0.01
  );

  // Edge case: zero seconds
  runner.assertAlmostEqual(
    'paceToMinutes("8:00") should return 8.0',
    paceToMinutes('8:00'), 8.0, 0.01
  );

  // Edge case: invalid input
  runner.assertEquals(
    'paceToMinutes("invalid") should return 0',
    paceToMinutes('invalid'), 0
  );
}

function testFormatPace(runner) {
  runner.assertEquals(
    'formatPace(6.5) should return "6:30"',
    formatPace(6.5), '6:30'
```

```
  );
  runner.assertEquals(
    'formatPace(5.0) should return "5:00"',
    formatPace(5.0), '5:00'
  );
  runner.assertEquals(
    'formatPace(7.25) should return "7:15"',
    formatPace(7.25), '7:15'
  );
}

function testPaceRoundTrip(runner) {
  // Integration test: paceToMinutes and formatPace are inverse operations
  var original = '6:30';
  var result = formatPace(paceToMinutes(original));
  runner.assertEquals(
    'Round-trip: formatPace(paceToMinutes("6:30")) should return "6:30"',
    result, original
  );

  original = '5:00';
  result = formatPace(paceToMinutes(original));
  runner.assertEquals(
    'Round-trip: formatPace(paceToMinutes("5:00")) should return "5:00"',
    result, original
  );
}

// ============================================================
// Unit Tests: statistical functions (HeartRateVsPace)
// ============================================================

function testCorrelation(runner) {
  // Create a temporary HeartRateVsPace instance to access its methods
  var vis = new HeartRateVsPace();

  // Perfect positive correlation
  var x1 = [1, 2, 3, 4, 5];
  var y1 = [2, 4, 6, 8, 10];
  runner.assertAlmostEqual(
    'Correlation of perfectly linear positive data should be 1.0',
    vis.calculateCorrelation(x1, y1), 1.0, 0.001
  );

  // Perfect negative correlation
  var y2 = [10, 8, 6, 4, 2];
  runner.assertAlmostEqual(
    'Correlation of perfectly linear negative data should be -1.0',
    vis.calculateCorrelation(x1, y2), -1.0, 0.001
  );

  // No correlation (orthogonal data)
  var x3 = [1, 2, 3, 4, 5];
  var y3 = [3, 3, 3, 3, 3];
  runner.assertAlmostEqual(
    'Correlation of constant Y data should be 0.0',
    vis.calculateCorrelation(x3, y3), 0.0, 0.001
  );
}

function testRegression(runner) {
  var vis = new HeartRateVsPace();

  // y = 2x + 1 → slope=2, intercept=1
  var x = [1, 2, 3, 4, 5];
  var y = [3, 5, 7, 9, 11];
```

```
  var result = vis.calculateRegression(x, y);

  runner.assertAlmostEqual(
    'Regression slope for y=2x+1 should be 2.0',
    result.slope, 2.0, 0.001
  );
  runner.assertAlmostEqual(
    'Regression intercept for y=2x+1 should be 1.0',
    result.intercept, 1.0, 0.001
  );

  // y = -0.5x + 10 → slope=-0.5, intercept=10
  var y2 = [9.5, 9, 8.5, 8, 7.5];
  var result2 = vis.calculateRegression(x, y2);

  runner.assertAlmostEqual(
    'Regression slope for y=-0.5x+10 should be -0.5',
    result2.slope, -0.5, 0.001
  );
  runner.assertAlmostEqual(
    'Regression intercept for y=-0.5x+10 should be 10.0',
    result2.intercept, 10.0, 0.001
  );
}

// ============================================================
// Unit Tests: moving average (PaceProgress)
// ============================================================

function testMovingAverage(runner) {
  var vis = new PaceProgress();

  var data = [10, 20, 30, 40, 50];

  // Window size 1: result should equal input
  var result1 = vis.calculateMovingAverage(data, 1);
  runner.assertAlmostEqual(
    'Moving average window=1: first element should be 10',
    result1[0], 10, 0.01
  );
  runner.assertAlmostEqual(
    'Moving average window=1: last element should be 50',
    result1[4], 50, 0.01
  );

  // Window size 3: trailing average
  var result3 = vis.calculateMovingAverage(data, 3);
  runner.assertAlmostEqual(
    'Moving average window=3: element[0] should be 10 (only 1 value)',
    result3[0], 10, 0.01
  );
  runner.assertAlmostEqual(
    'Moving average window=3: element[1] should be 15 (avg of 10,20)',
    result3[1], 15, 0.01
  );
  runner.assertAlmostEqual(
    'Moving average window=3: element[2] should be 20 (avg of 10,20,30)',
    result3[2], 20, 0.01
  );
  runner.assertAlmostEqual(
    'Moving average window=3: element[4] should be 40 (avg of 30,40,50)',
    result3[4], 40, 0.01
  );
}

// ============================================================
```

```
// Unit Tests: colour mapping (TrainingHeatmap)
// ============================================================

function testHeatmapColours(runner) {
  var vis = new TrainingHeatmap();
  var maxDist = 20;

  // No activity should return gray
  var c0 = vis.getDistanceColor(0, maxDist);
  runner.assertEquals(
    'Heatmap colour: 0km should be gray (235)',
    c0[0], 235
  );

  // Low distance (25% of max = 5km)
  var c1 = vis.getDistanceColor(4, maxDist);
  runner.assertEquals(
    'Heatmap colour: 4km/20km should be level 1 green (155)',
    c1[0], 155
  );

  // High distance (>75% of max)
  var c4 = vis.getDistanceColor(18, maxDist);
  runner.assertEquals(
    'Heatmap colour: 18km/20km should be level 4 dark green (33)',
    c4[0], 33
  );
}

// ============================================================
// Unit Tests: normalisation (MarathonRadar)
// ============================================================

function testRadarNormalisation(runner) {
  // Pace normalisation: 6:00 = 1.0, 9:00 = 0.0
  // Formula: max(0, min(1, 1 - (pace - 6.0) / 3.0))
  var score6 = Math.max(0, Math.min(1, 1 - (6.0 - 6.0) / 3.0));
  runner.assertAlmostEqual(
    'Radar: pace 6:00/km should normalise to 1.0',
    score6, 1.0, 0.01
  );

  var score75 = Math.max(0, Math.min(1, 1 - (7.5 - 6.0) / 3.0));
  runner.assertAlmostEqual(
    'Radar: pace 7:30/km should normalise to 0.5',
    score75, 0.5, 0.01
  );

  var score9 = Math.max(0, Math.min(1, 1 - (9.0 - 6.0) / 3.0));
  runner.assertAlmostEqual(
    'Radar: pace 9:00/km should normalise to 0.0',
    score9, 0.0, 0.01
  );

  // Distance normalisation: capped at 1.0
  var dist120 = Math.min(120 / 100, 1);
  runner.assertAlmostEqual(
    'Radar: 120km should cap at 1.0',
    dist120, 1.0, 0.01
  );
}

// ============================================================
// Run all tests
// ============================================================

```

```
function runAllTests() {
  var runner = new TestRunner();

  testPaceToMinutes(runner);
  testFormatPace(runner);
  testPaceRoundTrip(runner);
  testCorrelation(runner);
  testRegression(runner);
  testMovingAverage(runner);
  testHeatmapColours(runner);
  testRadarNormalisation(runner);

  return runner;
}
/* End of my own code */
```